DAY 11 · THEORY HANDOUT

LSTM: Keep Useful Context

Long Short-Term Memory: a recurrent network with a state path and learned gates.

KGP Talkie

Scan to subscribe

Laxmi Kant Tiwari · kgptalkie.com

1. Why add LSTM?

A simple RNN may lose a clue after many state updates. LSTM was designed to help with longer dependencies.

Early clue

"I do not think ..."

We need the word "not".

Words in between

"... the movie with music ..."

Other words arrive.

Later word

"... was fun."

The earlier clue matters.

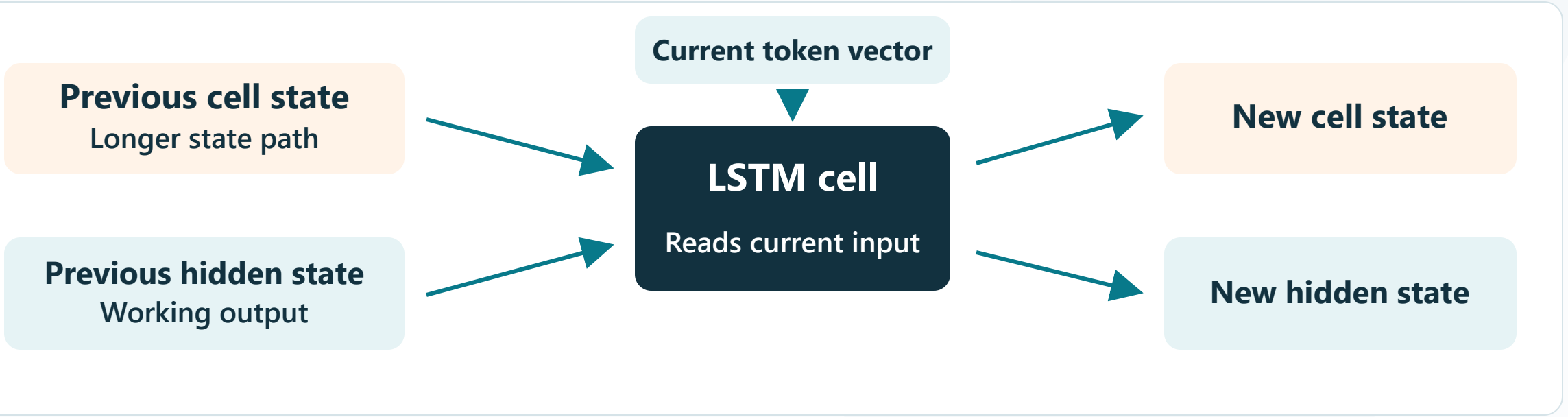
Context is earlier information that helps interpret later words. A **dependency** means one part of a sequence affects another.

LSTM stands for Long Short-Term Memory. It is a kind of RNN. It keeps an extra state path and controls changes using learned gates.

> LSTM can help with earlier clues. It does not guarantee that every long sentence or meaning is remembered.

2. Two states pass to the next step

The **cell state** is an extra path for carrying information. The **hidden state** is the current working output.



Both states are vectors: rows of learned numbers. The cell state can carry useful information through many steps with smaller changes.

The hidden state is passed to the next cell and can be used for predictions. Independent sentences begin with zero states in our project.

> These states are numbers, not copied words. "Memory" is a name for the state path, not human memory.

3. A gate controls how much passes

A **gate** is a set of values from 0 to 1 that controls parts of a state vector.

Gate near 0

Pass very little.

Mostly block this part.

Gate near 0.5

Pass part of it.

A partly open gate.

Gate near 1

Pass most of it.

Mostly keep this part.

The current token vector and previous hidden state are used to calculate gates. Learned weights and **sigmoid** produce values from 0 to 1.

The **forget gate** controls old information. The **input gate** controls new information. The **output gate** controls the working output.

> Training learns the gate behavior. We do not write rules such as "always keep the word not".

4. Forget gate: keep useful old information

The forget gate scales each part of the previous cell state.

Forget gate = scale each part of the old cell state



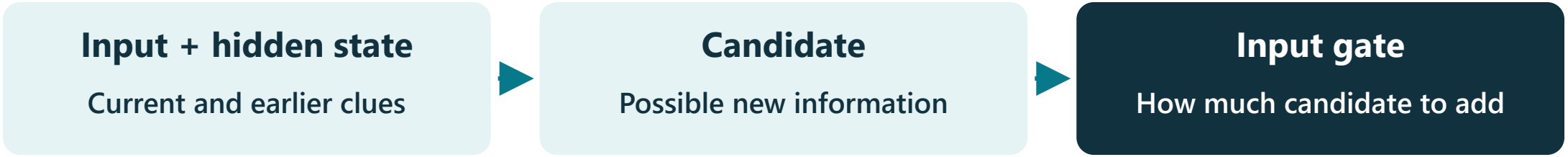
A gate value near 1 keeps most of that part. A value near 0 removes most. It may keep a useful earlier clue while reducing unrelated information.

There is one gate value per state component. Some parts can be kept while other parts are reduced at the same step.

> The name "forget" describes a numeric change. It does not mean that the model deletes a stored word.

5. Input gate: choose new information

A **candidate** is the proposed new state information made from the current input and previous hidden state.



Candidate values use tanh: each value stays between -1 and 1.

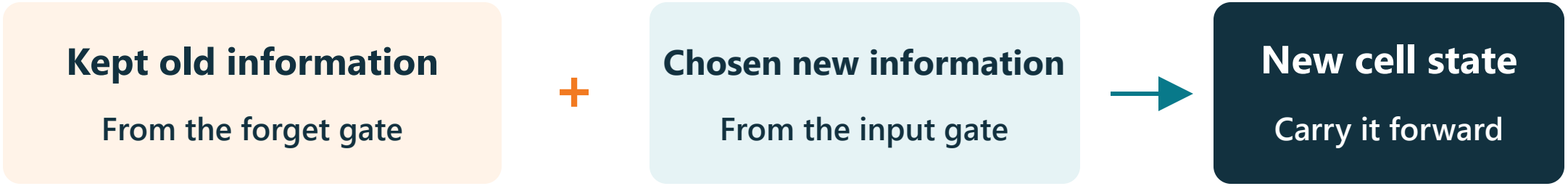
A learned calculation makes the candidate. **tanh** is an activation rule that keeps each candidate value between -1 and 1.

The input gate scales the candidate before it is added. Values near 0 add little; values near 1 add most.

> The candidate proposes what could be added. The input gate decides how much of each proposed part passes.

6. Update the cell state

Add the old information that survived and the new information allowed by the input gate.



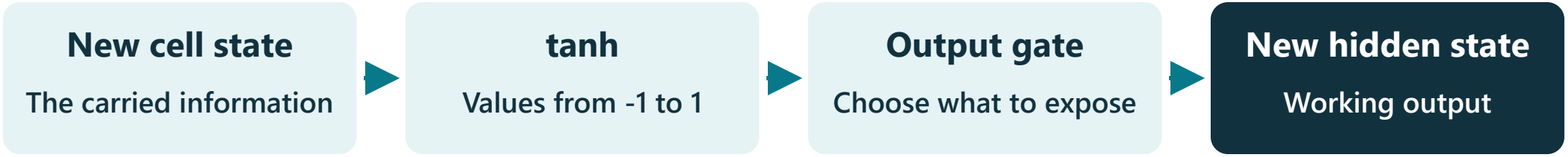
This state path makes it easier for useful information to pass through many steps. The gates control how much it changes.

An earlier negative clue can remain useful when later words arrive. This is a story about the task; the actual state contains numeric patterns.

> Keep old part + add new part = updated cell state. No full equation is needed to follow the flow.

7. Output gate: make the hidden state

The output gate controls which parts of the updated cell state form the new hidden state.



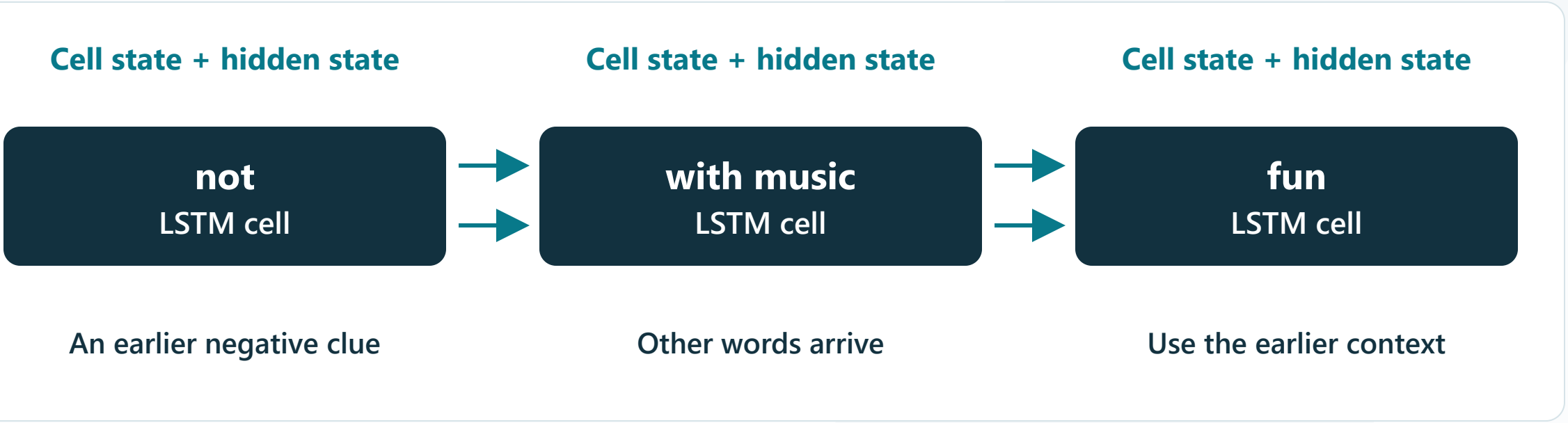
First, tanh bounds the updated cell values between -1 and 1. Then the output gate scales those values to form the hidden state.

Both the updated cell state and hidden state go to the next time step. For sentence classification, we use the last hidden state for a class prediction.

> The cell state carries information. The hidden state is the working output made available at this step.

8. Follow the sentence through time

"I do not think the movie with great music was fun." The early word "not" matters later.



Every real token gets its own step. The diagram shows three selected points. Each step uses the same LSTM cell and shared weights.

The gates may keep patterns linked to the earlier negation while processing later words. Training teaches what helps the final label.

> This is an intuition diagram, not an actual record of a particular cell value. LSTM can still misread language.

9. A small LSTM project

Use the same short sentence task to see how an LSTM layer fits into the text pipeline.



The notebook stands alone. It creates its own training and test sentences, trains the model, checks test accuracy, and predicts fresh inputs.

Backpropagation through time updates the embedding, LSTM weights, gates, and final layer. We use the same two-class loss as in the RNN lesson.

> A small score on this toy task does not prove an advantage over RNN. The project shows the LSTM steps in code.

DAY 11 · COMPLETE

Thank You

Follow KGP Talkie for new lessons, course updates and live sessions.

Scan to follow

Instagram
@kgptalkie

WhatsApp Channel
KGP Talkie